Republic of the Philippines
MARINDUQUE STATE UNIVERSITY
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
Tanza, Boac, Marinduque

PROJECT PROPOSAL: STUDENT COMMUTE ROUTE PLANNER

Team QUBITS

In Partial Fulfillment
of the Requirements for the Course
Data Structures and Algorithms

Members:
Ivyrhem Deinne M. Magararu
Alexis Aizer S. Peñaflorida
Herrah Athena I. Lunario
Riane Cyrish S. Ladaga
Jasmine E. Sapungan

Instructor:
ENGR. ALESSANDRA MAE P. MONLEON

November 12, 2026

PROJECT TITLE AND OVERVIEW

The Student Commute Route Planner is a Java-based desktop application designed to help students find the shortest and most convenient routes between different locations. Using the Graph Data Structure and Dijkstra's Algorithm, the system calculates the shortest path and provides estimated travel distance, travel time, and transportation fare. With a user-friendly interface developed using Java Swing, the application aims to make daily commuting easier, faster, and also provides convenient features that improve the overall commuting experience while demonstrating the practical application of data structures and algorithms.

PROBLEM STATEMENT

Current Issue

Students often rely on asking others, using social media, or depending on personal experience when planning their daily commute. These methods may consume time and do not always provide the shortest or most efficient route. Students may also forget previously searched routes and have to repeat the same search whenever they use the application. Without a simple route-planning system, commuting becomes less efficient, especially for new students or those unfamiliar with different locations.

Proposed Solution

The proposed Student Commute Route Planner addresses these problems by representing locations and roads as a weighted graph and applying Dijkstra's Algorithm to determine the shortest route between two selected locations. The application automatically calculates the most efficient route and displays the estimated distance, travel time, and fare. It also stores route history, favorite routes, and bookmarked locations using local file storage, allowing users to access them even after restarting the application. Additionally, the interactive graph visualization helps users easily understand how locations are connected and how the recommended route is determined.

PROJECT OBJECTIVES

The following objectives define the goals that the project intends to accomplish throughout its development and implementation.

The project aims to achieve the following objectives:

- Develop a Java-based Student Commute Route Planner using the Graph Data Structure.
- Implement Dijkstra's Algorithm to determine the shortest route between two selected locations.
- Display the estimated travel distance, travel time, and transportation fare.
- Allow users to save and manage their favorite routes.
- Allow users to bookmark frequently visited locations for quick access.
- Save previously searched routes through a Route History feature using local CSV file storage.
- Provide an interactive graph visualization that represents locations and roads using vertices and edges.
- Design a simple and user-friendly graphical user interface using Java Swing.
- Demonstrate the practical application of graph algorithms, data structures, and file handling in solving real-world transportation problems.

KEY FEATURES AND SCOPE

This section outlines the major features of the proposed system and defines the scope and limitations of the project to establish clear boundaries for its development.

Features

- Allows users to select their starting location and destination.
- Calculates and displays the shortest commuting route.
- Shows the total travel distance.
- Estimates travel time based on the selected route.
- Calculates the estimated transportation fare.
- Save frequently used routes as Favorite Routes.

- Bookmark commonly visited locations such as Home, School, or Terminal.
- Save previously searched routes through Route History.
- Interactive graph visualization using circles (vertices) and lines (edges), with the shortest route highlighted.
- Provides an easy-to-use graphical user interface for students.
- Supports searching and selecting available locations.

Out of Scope

The proposed system will not include real-time GPS navigation, live traffic monitoring, online maps, or public transportation schedules. It is intended as an educational desktop application that demonstrates the application of graph algorithms, data structures, and local file storage. Cloud synchronization, user accounts, and mobile application support are also outside the scope of this project.

PROPOSED STRUCTURED DATA

This section describes the data structures that will be used in the development of the system and explains how each one contributes to the efficient storage, organization, and processing of data.

Table 1

Data Structure	Implementation	Purpose	Justification
Graph (Adjacency List)	ArrayList of Vertices and Edges	Represents locations and connecting roads	Efficiently models the transportation network while minimizing memory usage.
Priority Queue (Min Heap)	Java PriorityQueue	Supports Dijkstra's Algorithm	Efficiently selects the nearest unvisited location.
HashMap	Key-Value Pair	Stores location names and bookmarked locations	Provides fast average O(1) lookup performance.
ArrayList	Dynamic Array	Stores route history, favorite routes, and graph data	Supports dynamic storage and efficient traversal.

PROPOSED ALGORITHM AND DATA MANIPULATION

This section discusses the algorithms and data manipulation techniques that will be implemented in the system. It also presents the time and space complexity of the major operations to evaluate their efficiency.

A. Data Manipulation

- **Traversal –** Traverses the graph to explore connected locations and identify possible routes between the selected starting point and destination.
- **Searching** – Searches for a specific location or destination entered by the user using a HashMap for fast and efficient retrieval of location information.
- **Insertion** – New locations (vertices) and roads (weighted edges) can be added to the graph, allowing the transportation network to expand and remain up to date.
- **Updating** – Existing location names, road information, or edge weights can be modified whenever changes occur in the transportation network.
- **Deletion** – Existing locations or roads can be removed from the graph, automatically updating the connections and maintaining the consistency of the route network.

B. Core Algorithms & Algorithmic Complexity

Table 2

Operation	Algorithm	Time Complexity	Space Complexity
Route Finding	Dijkstra's Algorithm	O((V+E) log V)	O(V)
Location Search	HashMap Search	O(1) average	O(1)
Graph Traversal	Breadth-First Search (BFS)	O(V+E)	O(V)
Add Road/Location	Adjacency List Insertion	O(1)	O(1)
Save Data	Java FileWriter (CSV)	O(n)	O(1)
Load Data	BufferedReader (CSV)	O(n)	O(n)

TOOLS AND TECHNOLOGY STACK

This section presents the software, programming language, development environment, and technologies that will be used to design, implement, and test the proposed application.

- **Programming Language: Java**

  Java will serve as the primary programming language for developing the application. It will handle system operations, route calculations, user inputs, and file management.

- **Development Environment: IntelliJ IDEA**

  IntelliJ IDEA will be used as the primary development environment for writing, editing, compiling, debugging, and testing the Java application.

- **GUI Framework: Java Swing**

  Java Swing will be used to create the application's graphical user interface, including forms, buttons, menus, and display panels.

- **Data Structures: Graph, HashMap, ArrayList, and PriorityQueue**

  These data structures will represent the transportation network, manage locations, store route information, and support efficient route computation.

- **Algorithms: Dijkstra's Algorithm and Breadth-First Search (BFS)**

  Dijkstra's Algorithm will determine the shortest route, while Breadth-First Search (BFS) will be used for graph traversal and exploring connected locations.

- **Storage: Local CSV Files using Java File I/O (FileWriter and BufferedReader)**

  Java File I/O will be used to save and retrieve route history, favorite routes, and bookmarked locations using CSV files.

- **Visualization: Java Swing Graphics (Graphics2D)**

  Java Swing Graphics will display the transportation network using circles for locations and connecting lines for roads, highlighting the selected shortest route.

DEVELOPMENT TIMELINE

This section provides the proposed schedule for the development of the project. It outlines the major tasks and milestones that will be completed during each phase of the project to ensure timely completion.

Table 3

Phase	**Task**	**Target**
Phase 1: Planning & System Design	Gather requirements, design the interface, graph structure, and data storage.	Week 2
Phase 2: Core Development	Implement the Graph Data Structure, Dijkstra's Algorithm, CRUD operations, and file saving/loading.	Week 2
Phase 3: User Interface & Testing	Develop the Java Swing interface, implement graph visualization, and perform system testing.	Week 3
Phase 4: Documentation & Final Presentation	Complete documentation, analyze algorithm complexity, perform final testing, and prepare the project presentation.	Week 4